GEMINI CLI:
A1 : Conceptual Architecture

Group Name : Group 25

CISC 322/326

Date : February 13, 2025

Video Link : https://youtu.be/bslDLN8hrIl

Group Names & Roles

Caravaggio, Emanuel - Presenter , Introduction, Conclusion, AI Collaboration (Student I.D : 20403106)

Clulow Damon - Abstract, Concurrency, Data Dictionary, Formatting (Student I.D : 20285634)

Dhungana, Raghab - Architectural Styles, Control and Data Flow (Student I.D : 20354596)

Elamin, Adam - User Use-Case, General Timeline (Student I.D : 20271708)

Fernando, Paul - Team Leader, Maintenance/Developer Workflow, Lessons Learnt (Student I.D : 20261647)

Shetty, Aditya - Presenter , Components and Interactions, Naming Conventions (Student I.D : 20320373)

Table Of Contents

Abstract - 4

System Overview - - - - - - - - - - - - - - - 5

Components and Interactions - - - - 6

Architecture Styles - - - - - - - - - - - - - 7

Control and Data Flow - - - - - - - - - - - 8

Concurrency - - - - - - - - - - - - - - - - - - - 9

External Interfaces - - - - - - - - - - - - - 10

Use Case #1 - - - - - - - - - - - - - - - - - - 11

User Case #2 - 12

System Evolution - 13

General Growth - 14

Maintainer / Developer Workflow - 15

Naming Convention - 16

Data Dictionary - 17

Lessons Learned - 18

AI Collaboration - 19

Conclusion - 20

Abstract

Gemini CLI is an open-source command-line AI tool that uses a client server architecture style divided into 3 main components

The system consists of three main modules :

- **CLI (GUI)** – handles user input/output
- **Core (Backend)** – constructs prompts with context, and communicates with the Gemini API and verifies installed tools
- **Tools Module** - enables file system, shell, and third-party service interactions via the Module Context Protocol

The system can be operated directly by the user or through the command line, capable of automated issue triage. This is the defining difference between a simple AI tool and Gemini CLI's capabilities.

System Overview

- As mentioned earlier Gemini Uses a **2-way client** server architecture with a layered component design.

- This is to separate interactive components, which allows for more robust maintainability, and allows servers to automate scaling procedures.

- Starting with a User Input, A textual input is sent to the Gemini API and the CORE manages processing of results as readable output. These actions are asynchronous as to keep the GUI responsive to User changes

Components and Interactions

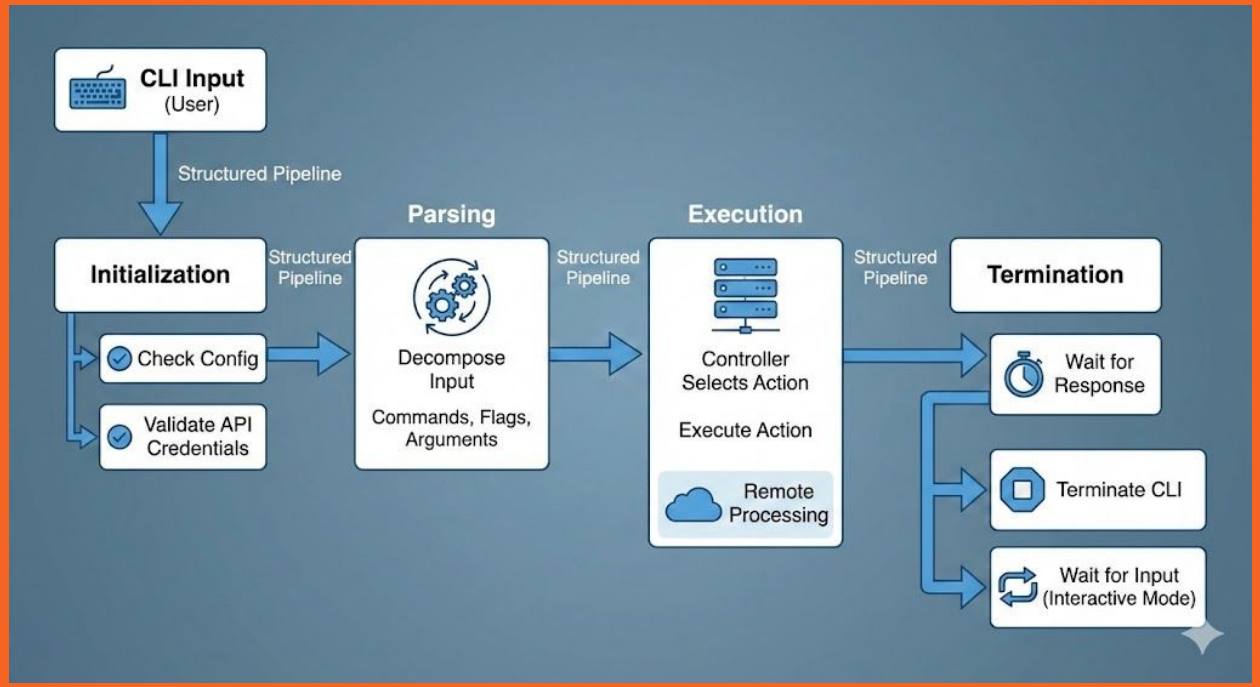
1) User or Command Line Input
   - ➢ A prompt is entered through the command line or by a user
2) CLI
   - ➢ The CLI checks for previous chat history or user setting configurations to influence the prompt
3) Core
   - ➢ The Core finalizes the prompt and sends it to the API for model reasoning
4) Verified Tools
   - ➢ User Requested or Pre-Registered User Tools are returned to the Core, so that they can be integrated as contextual reasoning for the prompt
5) Output Formatting
   - ➢ The Response is then formatted into an Image or as Text
6) CLI
   - ➢ The CLI then delivers the response to the User through the GUI

Architecture Styles

Gemini CLI, uses a 2-way client server style that can be divided into the following layers :

- **Presentation Layer :**
  - Translates human intent into structured data and turns raw API responses into readable text.
- **Application/Logic Layer :**
  - Translates human intent into structured data and turns raw API responses into readable text.
- **Network/Service Layers :**
  - Responsible for communication between the CLI and remote Gemini services.
- **Server-side (Remote) :**
  - Accounts for responsibilities managed by Google's infrastructure.

Control and Data Flow



Concurrency

Gemini's Concurrency is Asynchronous, so that the User can respond to operation with a longer latency

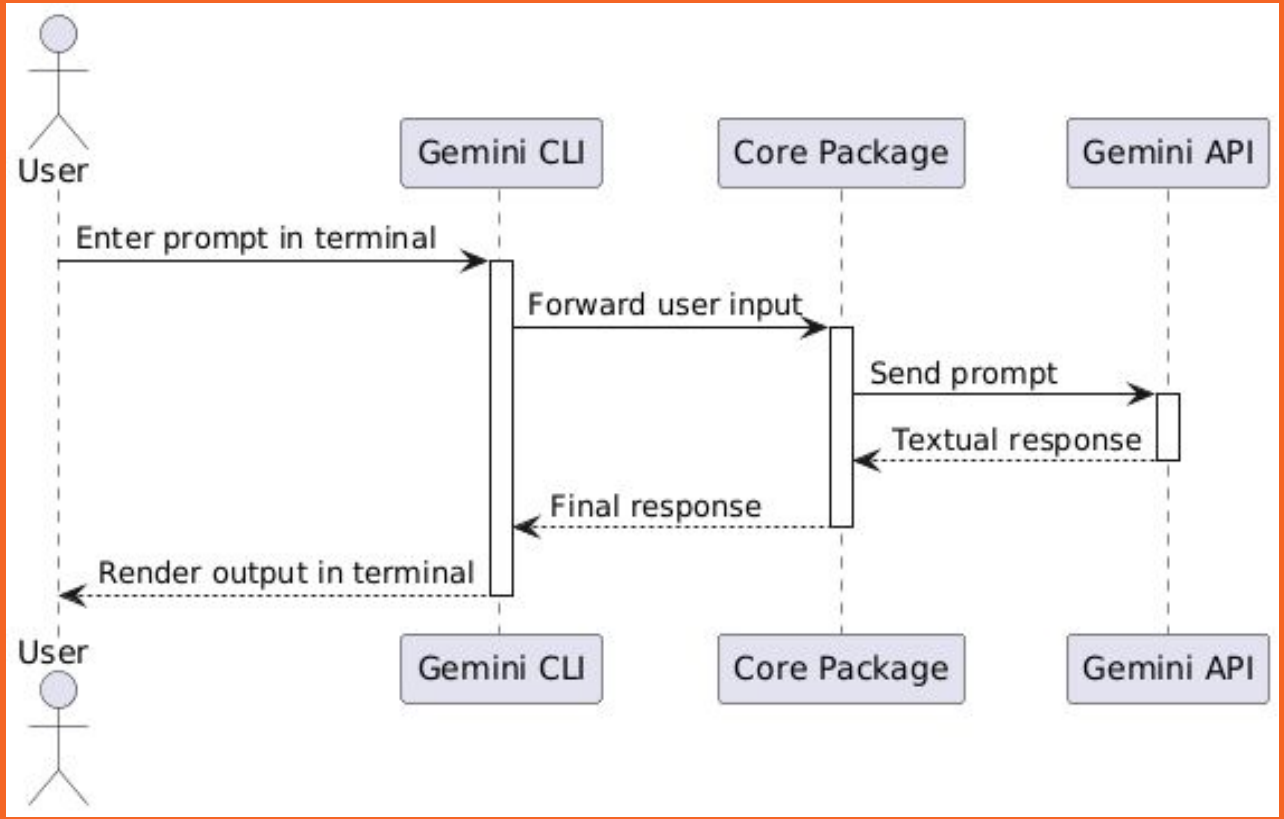
The main Concurrency Component is the CLI :

- The CLI takes user input and interacts with the Gemini API, which may branch into managing verified tools
- The execution of tools allows the model to interact with the local environment, including reading files and running shell commands, which further increases latency
- Prompts are then formatted and sent to the API, which is asynchronously displayed as the output.

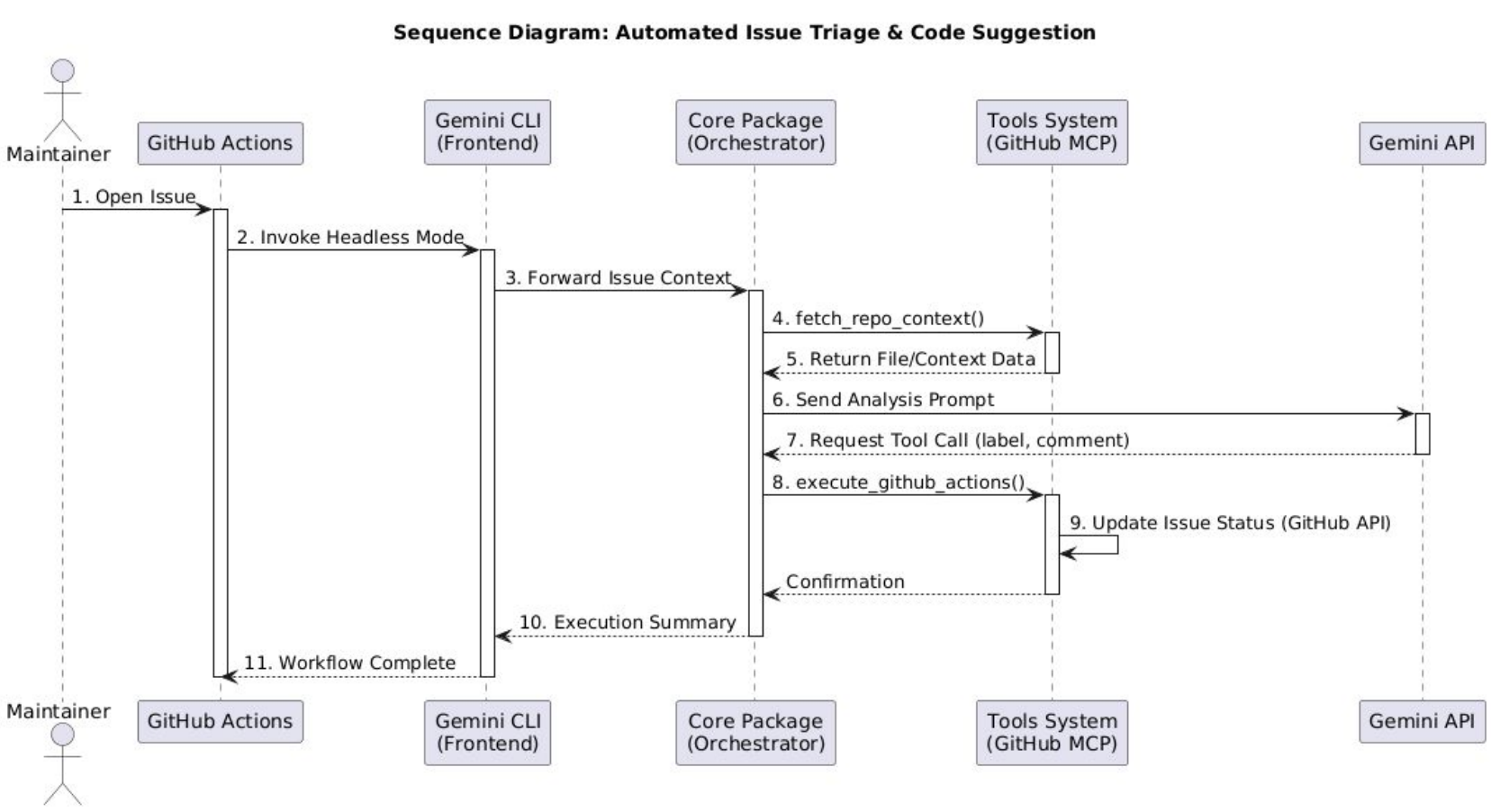
External Interfaces

1. User and Terminal Interface
   - Frontend for the LLM
   - Manages conversation history
   - Controls user preferences
2. File System and Shell Environment
   - Uses file system tools to maintain integration with the host
   - Utilizes glob and grep for discovering file patterns and searching content
3. Model Context Protocol and Extensions
   - Universal connector for GeminiCLI
   - Extension system allows developers to install pre-packaged integrations that bundle MCP configd and custom capabilities

Use Case 1: The user enters a prompt in the terminal



Use Case 2: Automated Issue Triage and Code Suggestion



General Growth

Growth of the Gemini CLI, from a simple developer tool, into a complete system for independent software work has come from a few key changes

1) The **Addition of the MCP,** which connected the system to other programs, including **Slack and GitHub** for standard setup
2) **Designing the AI to act like an agent** rather than a developer tool, using logical loops like Todo Planning to create a step-by-step plan for large requests
3) **Sequential Approval**, which is a safety check that keeps you in charge by asking for your okay before the AI makes sensitive changes

Maintainer / Developer Workflow

Maintainers and Developers must follow a standardized Node.js workflow.

1) Developers must use the Correct version of Node.js to avoid conflicts, and must incorporate a script which automates formatting, linting and unit tests.

2) Contributors fork (create a separate) the repository and make changes specific to the three key components enforcing division between each.

3) Finally, the "Dogfooding" approach, means using the system itself to make self corrections, which means using the CLI as a correction tool for maintenance.

Naming Convention

- **Gemini CLI:** An open-source AI agent accessible from the command line that accepts user prompts and calls the Gemini model.
- **CLI Package:** Presentation layer responsible for the user-facing portion of the Gemini CLI.
- **Core Package:** System orchestrator responsible for constructing prompts, managing system state, executing tools and interacting with Gemini.
- **Tool Module:** Tools that enable the system to interact with the local environment through file and shell operations.
- **Node.js:** An open-source JavaScript runtime environment.
- **Gemini API:** Remote service that is used to pass prompts to Google's Gemini models and receive responses.
- **Token:** Single unit of data that is processed by an AI model during inference.
- **Model Context Protocol (MCP):** Open-source standard for AI applications to connect with third-party services.

Data Dictionary

1) Tools - Verified tools that extend the capabilities of the Current Model
   - Including - read_file, write_file, web_search

2) **Slash Commands -** Commands that the User can Use to Interact with the CLI.
   - Including - /help, /chat, /memory

3) **Directory Structure -** Organizes the Code Base into Distinct Components
   - Including - read_file, write_file, web_search

Lessons Learned

- Learned the importance of separating the user interface from the core logic to make the system more flexible and easier to update
- Understood how asynchronous design keeps the system responsive while waiting for slow API responses
- Identified network latency and dependence on remote APIs as key performance challenges
- Recognized the need for strong architecture, automation, and testing to support stable and scalable development.

AI Collaboration

- Used ChatGPT - 5.2, February 2026 version, as well as Google's Gemini 3, November 2025 version
- Designated specific tasks that our AI member could perform
- Providing valid, fact-checked information to the system prior to prompting and cross-referenced LLM outputs to ensure factual errors, bias and hallucinations did not occur
- Contributed around 10% of the final deliverable, a significant amount but did not contribute by any means more than any member of our team
- Learned to apply AI as a tool to further and validate understanding rather than provide answers

Gemini

Conclusions

- Powerful platform for interacting with Google’s Gemini using user focused command line interface
- Hybrid architecture combining Client-Server and Layered styles
- CLI package, Core package and Tool subsystem operate as integrated components while remaining distinct
- Defined control and data flows with asynchronous concurrency mechanisms
- Use of MCP and developer workflow make the system's adaptability a key feature